

ПОЗИЦИЯ

8 СЕНТЯБРЯ 2025 ГОДА

Письмо высшему руководству: меняющаяся роль менеджеров по работе с ключевыми клиентами

Ваши менеджеры по работе с ключевыми клиентами — замечательные люди, но вы поручаете им не ту работу.



Чарльз О'Нил (Базеттен)



Мудрит Джаясекара (Басетен)

Если вы нанимаете команду инженеров в компанию, специализирующуюся на искусственном интеллекте, в таких областях, как здравоохранение, страхование, юридические технологии или финансы, то, скорее всего, вы попали в ловушку, в которую попадают многие: вы считаете, что ваши специалисты по машинному обучению могут работать и как инженеры по большим языковым моделям.

Проблема не в том, что они недостаточно умны. Проблема в том, что вы заставляете их делать не то, что нужно. Их задача не в том, чтобы заново изобретать обучение предельных моделей в вашем стартапе. Их задача — помочь вам выйти на соответствие продукта рынку.

Роль MLE в создании прототипов

Это не значит, что MLE не играют важной роли. На самом деле зачастую именно они лучше всего подходят для создания прототипов на ранних этапах.

Они могут подключать API, проводить небольшие эксперименты, чтобы подготовить вас к первым демонстрационным версиям. На этом этапе главное — скорость, итерации и поиск соответствия между проблемой и решением. Малые и средние предприятия выигрывают в этом плане, потому что они гибкие, ориентируются на количественные показатели и могут быстро разработать прототип.

Но как только вы убедились в наличии проблемы и увидели первые признаки соответствия продукта рынку, ситуация меняется. На этом этапе вам нужно перейти от наспех собранных демоверсий к масштабируемым и надежным циклам после обучения. И это не та задача, которую могут или должны решать ваши MLE. Здесь на помощь приходит Baseten.

Несоответствие интеллекта

Ошибочно приравнивать навыки, которые делают человека хорошим специалистом в области машинного обучения, к навыкам, необходимым для управления командой разработчиков, использующих большие языковые модели. Докторская степень в области нейронных тангенциальных ядер и победы в соревнованиях Kaggle — это не то, что вам действительно нужно.

Даже если вы просто обращаетесь к моделям с закрытым исходным кодом через API (не говоря уже об обучении и логическом выводе), большие языковые модели требуют принципиально иного подхода к интеллекту. Чтобы добиться высокой производительности, нужно не просто использовать модели с градиентным бустингом. Нужно нечто более размытое, более качественное.

Для написания оценочных инструментов, использующих LLM в качестве судьи, требуется особая интуиция, которая не всегда согласуется с традиционными показателями количественного интеллекта. (Вот почему мне кажется, что такие компании, как Cognition, хвастающиеся тем, что нанимают победителей Международной математической олимпиады, идут по ложному пути.) Решение задач Международной математической олимпиады — это впечатляющий навык, но он совершенно не похож на интуитивное понимание того, как большие языковые модели обрабатывают токены. Как особенности английского языка как механизма формирования привычек приводят к тем едва заметным, но пагубным проявлениям, которые мы все так хорошо знаем и любим.

Ловушка обучения

«Но ведь именно в этом и заключается преимущество традиционного MLE, — подумаете вы, — в обучении и ML Ops в больших масштабах?»

Не совсем.

Обучение больших языковых моделей — это не просто разделение на обучающую и тестовую выборки и поиск гиперпараметров, на котором набивают руку гурь Kaggle. Успешное обучение или доработка языковой модели во многом зависят от качества данных. А это, опять же, требует качественного интеллекта: способности глубоко

понимать проблему и требования, предъявляемые к ее решению, определяя ее на естественном языке, а не с помощью детерминированного кода.

Когда вы дообучаете модель на основе результатов более крупной модели, а после обучения `qwep-32b` вы разочарованы из-за разрыва в дистилляции, куда вам податься? Генерация более качественных данных требует творческого подхода. А обучение с подкреплением — это вообще отдельная история. Как написать систему оценки, которая не ограничивается регулярными выражениями и детерминированными проверками на Python и действительно может служить полезной функцией вознаграждения? Как вообще понять, что ваша LLM в роли судьи оценивает именно то, что вам нужно, согласуется ли она с мнением экспертов в предметной области и достаточно ли четко определена для обучения с подкреплением?

Обучение LLM принципиально сложнее, чем раскручивание модели в Sklearn. Модели на *много* порядков больше. Обучение требует таких знаний о распараллеливании, сегментировании и распределенных системах, каких раньше никогда не было. Ожидать, что ваш MLE освоит все это, одновременно изучая принципиально иную парадигму взаимодействия с LLM и управления ими, — огромная задача. Нельзя просто так отнести специалиста по машинному обучению к категории «инженера по промт-инжинирингу + гуру аппаратного обеспечения + эксперта по распределенным системам + гуру глубоких трансформеров», если его последняя работа заключалась в обучении сверточных нейронных сетей на основе данных оптической когерентной томографии.

Ваша команда должна тратить время на определение проблемы, а не на решение инженерных задач, связанных с обучением моделей. Роль Baseten заключается в том, чтобы взять на себя эту инженерную нагрузку и предоставить готовые к использованию модели, соответствующие вашим требованиям.

The social intelligence factor

Perhaps most critically, LLM engineering demands social and personable intelligence in ways traditional ML never did.

Tasks don't exist in isolated boxes anymore. They're not completely defined by the features and labels you train on. Most LLM tasks are icebergs, with massive underwater components of domain knowledge and implicit context only available to teams expert in your specific vertical.

LLM engineers need to be socially intelligent enough not just to grasp this domain context, but to engage productively with domain experts and end users. They need to hold complex, nuanced discussions with stakeholders and distill all that messy, implicit information into evaluations, generation prompts, and training data.

Baseten takes those definitions and runs the heavy machinery: post-training, evaluation harnessing, RL, infrastructure.

The frontier problem

Perhaps the most salient issue is that the frontier of LLM research moves so quickly that it's impossible for engineers hired for specific vertical use cases to also position themselves on the rapidly expanding frontier.

No offence, but you're not getting Big Lab-level engineering quality at your app layer company. There's a reason the best talent goes to the big labs: They're building AGI, which is sexy. They get to work on fundamental language model problems. They pay better. They don't have to optimise a system for medical record data summarisation and JSON formatting.

To expect these MLEs to perform at a level where they can post-train the way big labs do, managing these systems at scale, is crazy. Everything is so new that the best commodity to hire for is raw intelligence and horsepower. Experience means less as things move faster. A PhD in convex optimisation doesn't really give you any relevant experience in spinning up a GRPO run for the first time.

The case for specialisation

All of this strengthens the argument for specialisation and buying versus building.

In addition, every interaction with teams in these verticals has only strengthened our conviction. The most self-aware companies understand this: They want Big Lab expertise packaged into models and tools their engineers can actually use productively. Sure, some are happy calling closed-source APIs forever. But anyone who wants a cheaper, faster, and most importantly, better model optimised for their specific task will eventually have to bite the bullet on post-training.

That's not to say tools and frameworks won't emerge that allow in-house engineering teams to actively participate in this process. Instead, we want to commoditise Big Lab talent so anyone can access it as a nicely packaged product.

But just putting a UI around a finetuning service isn't the answer. Yes, writing finetuning code is difficult, and automating it solves that problem. But it's only a tiny piece. The hard part about LLMs is that optimising them requires understanding everything end-to-end: from your generation prompt to your LLM-as-a-judge, through the complex mechanics of RL and SFT, and how to properly integrate all the feedback signals into a system that actually improves at what you want it to improve at.

Bottom line

If after all this, you still trust your MLEs to achieve what you want, then good for you. But you wouldn't build your own server rack with a team of software engineers who only know Java and API development. So why would LLMs be any different?

Other research



POSITION

The Michael Scott Paper Company of AI

Open-source doesn't need to wait out the frontier labs. It just has to wait out the gap.

 CHARLES O'NEILL



POSITION

Continual learning and the post monolith AI era

The cost of continual learning scales with generality.



CHARLES O'NEILL • 1 other

POSITION

If we can't design neat latent structures, then maybe we can Bitter Lesson it through self-study

They very existence of Claude's Constitution shows that self-study, to a certain extent, just works.



CHARLES O'NEILL

Explore Baseten today

START DEPLOYING

TALK TO AN ENGINEER >

Product

Dedicated Inference

Model APIs

Training

Frontier Gateway

Baseten Inference Platform

Model Runtimes

Infrastructure

Multi-cloud

Deployment options

Cloud

Self-hosted

Hybrid

Embedded engineering

Forward deployed engineers

Modalities

Transcription

Image Generation

Text-to-speech

Large language models

Compound AI

Embeddings

Industries

Enterprise

Healthcare

Developer

Model library

Documentation

Changelog

Resources

Research

Customers

Blog

Guides

Events

Partners

Savings calculator

Trust Center

About us

Startups

Careers

Contact us

Popular models

GLM 5.2

Kimi K2.7 Code

DeepSeek V4

GPT OSS 120B

Whisper Large V3

NVIDIA Nemotron 3 Ultra

Explore all

Legal

Terms and Conditions

Privacy Policy

Service Level Agreement

● ALL SYSTEMS NORMAL

© 2026 BASETEN

